

ПРИЛОЖЕНИЕ А

Теория двухконтурной архитектуры

Словарь терминов статьи и рекомендации с пометкой обоснованности

Это «мост» между простым языком статьи и строгими понятиями. Каждый термин статьи здесь раскрыт (§1), а каждая рекомендация помечена по статусу — что именно **доказано теорией**, а что является инженерным следствием или нормой. Цель — чтобы читатель не путал доказанный факт с разумной практикой.

Как читать пометки:

[ФАКТ] — математически доказанное утверждение или прямое следствие определения.

[СЛЕДСТВИЕ] — логически следует из факта при явно названном условии.

[РЕКОМЕНДАЦИЯ] — инженерная практика, согласованная с теорией и нормами, но не являющаяся теоремой.

[ЭМПИРИКА] — подтверждено опубликованными экспериментами, а не доказано.

[НОРМА] — требование стандарта или закона.

[НЕ ТЕОРЕМА] — часто встречающееся утверждение, которое теория НЕ доказывает (важно не выдавать за доказанное).

Связь документов. Статья даёт интуицию и таблицы решений; это приложение — обоснования; Приложение В — индустриальный пример (Siemens × NVIDIA). Термины во всех трёх совпадают: см. словарь §1.

Содержание

1. Словарь: термин статьи → строгое понятие → где раскрыт

Левый столбец — слова, которыми пользуется статья. Средний — строгое понятие. Правый — раздел этого приложения.

Термин в статье	Что это строго	Раздел
«угадывающая» модель, нейросеть	вероятностная модель: минимизирует ожидание потерь	§3
открытый контур	слой генерации и предложений, без доступа к оборудованию	§3
замкнутый контур	слой детерминированной проверки допустимости	§4, §8
жёсткое требование («так нельзя никогда»)	robust-ограничение: $g(x, \xi) \leq 0$ для всех $\xi \in U$	§2
мягкое требование («чем лучше, тем лучше»)	chance-ограничение / минимизация ожидания (хвост ϵ допустим)	§2, §3
«в среднем правильно» / «правильно всегда»	ожидание $E[\cdot]$ против худшего случая $\forall$	§2
забор жёстких ограничений	набор robust-ограничений, проверяемых детерминированно	§4
программа-сторож, проверяющий модуль	надёжный проверяющий (sound-оракул)	§8
виртуальная модель завода, цифровой двойник	дорогой sound-оракул (физическая симуляция)	§8, Прил. В
приблизительная оценка с честной пометкой	честная мера уверенности (conformal prediction)	§6
эталонные примеры / эталонные проверки	проверка самих правил (golden-тесты) — класс В3	§8
автоматический разбор текста программы	анализ дерева разбора (синтаксический анализатор)	§8
наблюдение в работе	контроль во время исполнения	§6, §8
запрет по умолчанию	нет проверки / проверка не пройдена → заблокировать	§8
безопасный отказ	отказ самого контура → безопасное состояние	§8, §10
классы (для теоретиков)	В1 / В2 / В3 — три ветви для жёстких требований	§8

2. Две формы требований (раскрытие «жёсткое / мягкое»)

Пусть x — решение (установка, настройка, фрагмент кода), ξ — неопределённый параметр (давление, задержка, входные данные), U — множество допустимых сценариев, $g(x, \xi) \leq 0$ — ограничение.

Тип	Строго	Термин в статье
robust (жёсткое)	$g(x, \xi) \leq 0$ для всех $\xi \in U$ (худший случай)	«так нельзя никогда»
chance (мягкое)	$P[g(x, \xi) \leq 0] \geq 1 - \varepsilon$ (хвост ε допустим)	«чем лучше, тем лучше»

[ФАКТ] Хвост ε неустраим данными. При $\varepsilon > 0$ chance-ограничение по построению допускает нарушение в доле случаев ε ; никакой объём обучающих данных и никакая функция потерь не превращают его в robust-ограничение. Это разница между утверждением о вероятности и утверждением «для всех».

[СЛЕДСТВИЕ] Жёсткое требование (по определению — выполняется в каждом случае) нельзя обеспечить только вероятностной моделью. Нужен слой, дающий поточечную гарантию для конкретного x .

3. Открытый контур (раскрытие «угадывающая модель», «в среднем»)

Открытый контур оптимизирует среднее: минимизирует $E_{\xi}[L(x, \xi)]$ при $P[g \leq 0] \geq 1 - \varepsilon$. Аппарат — chance-constrained programming (Charnes & Cooper, 1959; выпуклые аппроксимации — Nemirovski & Shapiro, 2006).

[ФАКТ] Обучение нейросети минимизирует ожидание потерь — это утверждение про среднее по распределению, а не про худший случай.

[РЕКОМЕНДАЦИЯ] Долю допустимых нарушений ε задавать осознанно, как проектный параметр, а не принимать «точность модели на тесте» за ε .

4. Замкнутый контур и цена робастности (раскрытие «забор»)

Замкнутый контур требует допустимости при любом $\xi \in U$ — это robust optimization. Soyster (1973) дал предельно консервативную форму; Ben-Tal & Nemirovski и El Ghaoui — вычислимые модели; Bertsimas & Sim (2004) ввели «бюджет неопределённости» Γ , плавно регулирующий консерватизм (вероятность нарушения $\approx \exp(-\Gamma^2/2n)$).

[ФАКТ] robust-оптимизация гарантирует допустимость для всех $\xi \in U$; за это платят ухудшением среднего результата — доказанный трейд-офф, «цена робастности» (Bertsimas & Sim, 2004).

[РЕКОМЕНДАЦИЯ] «Забор» — жёсткие ограничения — задавать явно и проверять для каждого изменения, заранее принимая, что среднее станет чуть хуже. Это плата за гарантию, а не дефект.

5. Язык «жёсткое / мягкое»: semiring-CSP (с честной оговоркой)

Чтобы держать жёсткие и мягкие требования в одной схеме, не смешивая их, удобен semiring-based CSP (Bistarelli, Montanari & Rossi, 1997).

Полукольцо	Что моделирует
({Да, Нет}, ИЛИ, И)	классическое жёсткое да/нет

Полукольцо	Что моделирует
$([0,1], \max, \min)$	мягкое: степень удовлетворения
$(\text{числа} \geq 0, \min, +)$	мягкое: стоимость/штраф

[ФАКТ] Полукольцо позволяет в одной формальной схеме держать жёсткие (да/нет) и мягкие (степень) требования, не теряя различия между ними.

[НЕ ТЕОРЕМА] Полукольцо **не доказывает** тезис «жёсткое $\rightarrow$ детерминизм». Оно даёт только язык. Решение «жёсткое отдаём детерминированному контуру» — инженерный шаг, и он опирается на наличие дешёвого надёжного проверяющего (§8), а не на полукольцо. Польза полукольца — защита от ошибки: не превращать запрет в «штраф с большим весом», который оптимизатор однажды перевесит.

6. Честная уверенность (раскрытие «приблизительная оценка с честной пометкой»)

Когда дешёвого надёжного проверяющего нет (класс B2), нужна честно измеренная уверенность. Conformal prediction (Vovk, Gammerman & Shafer, 2005; практическое введение — Angelopoulos & Bates, 2023) даёт множество ответов, накрывающее истину с заданной вероятностью.

Как это считается по шагам:

- на калибровочной выборке n считаем баллы несоответствия $s_i = |y_i - f(x_i)|$;
- берём порог $q = [(n+1)(1-\delta)]$ -й по возрастанию ($n = 1000$, $\delta = 0.1 \rightarrow 901$ -й из 1000);
- ответ: $C(x) = [f(x) - q, f(x) + q]$.

[ФАКТ] При обмениваемости данных множество $C(x)$ накрывает истину с вероятностью $\geq 1 - \delta$ — доказано, без предположений о виде распределения.

[ФАКТ] При сдвиге распределения (новая партия сырья, смена режима) гарантия теряется; восстанавливается только взвешенными версиями при известном отношении плотностей (Tibshirani et al., 2019; Barber et al., 2023).

[РЕКОМЕНДАЦИЯ] «Честную пометку» обязательно сопровождать контролем сдвига распределения. Без него пометка перестаёт быть честной — это прямое следствие предыдущего факта.

7. Когда ИИ действует, а не советует (constrained MDP, safe RL, shielding)

- **Constrained MDP** (Altman, 1999) — политика удерживает ожидаемую стоимость нарушений ниже порога (мягкий уровень).
- **Constrained Policy Optimization** (Achiam et al., 2017) — обучение с приближённым удержанием ограничений.
- **Shielding** (Alshiekh et al., 2018) — «щит» перехватывает каждое действие и корректирует только небезопасные.

[ФАКТ] Shielding обеспечивает поточечную блокировку небезопасного действия по формальной спецификации — это доказанное свойство метода.

[СЛЕДСТВИЕ] Если ИИ выбирает действия (а не только советует), жёсткий слой должен стоять на уровне действий (щит), а не только в обучении.

8. Надёжный проверяющий и классы V1/V2/V3

Надёжный проверяющий (sound-оракул) — инструмент, дающий вердикт «х допустим» со свойством *надёжности (soundness)*: ложных «ОК» не бывает (если сказал «ОК» — ограничение действительно выполнено). В статье это «программа-сторож». Дешёвые проверяющие: анализатор текста программы, проверка типов, диапазонов, шаблон, онтология. Дорогой — цифровой двойник.

[ФАКТ] Надёжность (soundness) — отсутствие ложных «ОК» — это свойство конкретного проверяющего, которое можно доказать (например, для проверки диапазона значений). Нас интересует именно soundness, а не полнота: безопасно иногда ложно забраковать корректное х.

Три ветви для жёстких требований (то, что в статье названо «классами для теоретиков»):

Класс	Условие	В статье	Гарантия
V1	есть дешёвый надёжный проверяющий	«программа-сторож»	поточечная (если проверяющий надёжен)
V2	проверяющего нет / он дорог	«оценка с честной пометкой»	честная, без гарантии
V3	правило само может устареть	«эталонные проверки»	соблюдения правила, не его истинности

[ФАКТ] Различие V1/V2/V3 — это разные типы гарантии, и они не взаимозаменяемы: V2 принципиально не даёт гарантии, V3 гарантирует соблюдение правила, но не его правильность.

[РЕКОМЕНДАЦИЯ] **Запрет по умолчанию:** нет проверки или проверяющий недоступен → блокировать, а не пропускать. Это практика, согласованная с нормами безопасности, а не теорема оптимизации.

[НОРМА] **Безопасный отказ:** отказ самого контура проверки должен переводить систему в безопасное состояние. Это требование функциональной безопасности (§10), а не оптимизационный результат.

9. Подтверждение паттерна в литературе по контроллерам

Двухконтурная идея встречается в свежих работах по генерации кода для контроллеров: нейросеть (открытый контур) обкладывают внешней проверкой (замкнутый).

[ЭМПИРИКА] LLM4PLC (Fakih et al., 2024) использует проверочную программу nuXmv; Agents4PLC сообщает до ~68.8% успешной формальной проверки на 23 задачах; работы по генерации с обратной связью показывают рост качества. LLM4SFC

формулирует ключевое: правильный синтаксис не гарантирует безопасность во время исполнения. Это подтверждение паттерна экспериментами, а не доказательство.

10. Привязка к нормам безопасности

Замкнутый контур обязан быть проверкой, совместимой со стандартами (аудируемые правила, явное выделение функций безопасности, безопасный отказ).

10.1. IEC 61508 — уровни полноты безопасности (SIL, режим низкого запроса)

SIL	Вероятность отказа на запрос (PFDavg)	Снижение риска (RRF)
SIL 4	$\geq 10^{-5} \dots < 10^{-4}$	100 000 ... 10 000
SIL 3	$\geq 10^{-4} \dots < 10^{-3}$	10 000 ... 1 000
SIL 2	$\geq 10^{-3} \dots < 10^{-2}$	1 000 ... 100
SIL 1	$\geq 10^{-2} \dots < 10^{-1}$	100 ... 10

10.2. ISO 13849-1 — уровни производительности (отказ/час)

Уровень	Средняя вероятность опасного отказа (1/ч)	Грубое соответствие SIL
a	$\geq 10^{-5} \dots < 10^{-4}$	—
b / c	$\geq 3 \cdot 10^{-6} \dots < 10^{-5} / \geq 10^{-6} \dots < 3 \cdot 10^{-6}$	SIL 1
d	$\geq 10^{-7} \dots < 10^{-6}$	SIL 2
e	$\geq 10^{-8} \dots < 10^{-7}$	SIL 3

[НОРМА] Прочие опоры: IEC 61511 (приборная безопасность процессов), IEC 62443 (защита АСУ ТП от вмешательства — важно для изолированного контура).

[ФАКТ] «99.9% на тесте» — это не уровень безопасности. Средняя точность измеряется по распределению, а уровень безопасности — это доказуемая граница вероятности опасного отказа конкретной функции при запросе. Это величины из разных пространств.

[НОРМА] Европейский закон об ИИ (Регламент ЕС 2024/1689, ст. 6) проводит ту же границу: ИИ, который только подсказывает, оптимизирует и повышает эффективность, не считается «компонентом безопасности» — пока его сбой не угрожает людям; функция, отвечающая за безопасность, считается. Это деление на жёсткое и мягкое, закреплённое в законе.

11. Сводная таблица: рекомендации и их обоснованность

Главная таблица приложения. Каждая рекомендация — с пометкой, **чем именно она подтверждена**. Что не доказано теорией — помечено отдельно.

Утверждение / рекомендация	Статус	Основание
Жёсткое требование нельзя обеспечить только вероятностной моделью	[ФАКТ]	хвост ϵ неустраним (§2)

Утверждение / рекомендация	Статус	Основание
Для жёсткого требования при наличии дешёвой надёжной проверки — детерминированный контур (B1)	[РЕКОМ.]	следствие при условии «есть надёжный проверяющий» (§8)
Нет дешёвой проверки → честная оценка с пометкой (B2), но без гарантии	[ФАКТ]	conformal не даёт гарантии (§6)
Правило может устареть → проверять само правило (B3)	[РЕКОМ.]	различие соблюдение/истинность (§8)
Запрет по умолчанию (нет проверки → блок)	[РЕКОМ.]	практика безопасности, не теорема
Безопасный отказ контура проверки	[НОРМА]	IEC 61508 (§10)
Conformal без контроля сдвига недостоверен	[ФАКТ]	нарушение обмениваемости (§6)
В сценариях безопасности ИИ — только в открытом контуре	[РЕКОМ.]	согласовано с §2 и законом (§10)
«Полукольцо доказывает: жёсткое → детерминизм»	[НЕ ТЕОР.]	полукольцо даёт только язык (§5)

12. Источники

1. Bertsimas, D., Sim, M. (2004). The Price of Robustness. Operations Research 52(1):35–53. DOI: 10.1287/opre.1030.0065.
2. Ben-Tal, A., Nemirovski, A. (1998). Robust Convex Optimization. Math. of OR 23(4):769–805.
3. Bertsimas, D., Brown, D. B., Caramanis, C. (2011). Theory and Applications of Robust Optimization. SIAM Review 53(3):464–501.
4. Charnes, A., Cooper, W. W. (1959). Chance-Constrained Programming. Management Science 6(1):73–79.
5. Nemirovski, A., Shapiro, A. (2006). Convex Approximations of Chance Constrained Programs. SIAM J. Optim. 17(4):969–996.
6. Bistarelli, S., Montanari, U., Rossi, F. (1997). Semiring-Based Constraint Solving and Optimization. J. ACM 44(2):201–236. DOI: 10.1145/256303.256306.
7. Vovk, V., Gammerman, A., Shafer, G. (2005). Algorithmic Learning in a Random World. Springer.
8. Angelopoulos, A. N., Bates, S. (2023). Conformal Prediction: A Gentle Introduction. FnT in ML 16(4):494–591. DOI: 10.1561/22000000101 (arXiv:2107.07511).
9. Tibshirani, R. J. et al. (2019). Conformal Prediction Under Covariate Shift. NeurIPS. arXiv:1904.06019.
10. Barber, R. F. et al. (2023). Conformal Prediction Beyond Exchangeability. Annals of Statistics 51(2):816–845.
11. Altman, E. (1999). Constrained Markov Decision Processes. Chapman & Hall/CRC.
12. Achiam, J. et al. (2017). Constrained Policy Optimization. ICML, PMLR 70:22–31. arXiv:1705.10528.
13. Alshiekh, M. et al. (2018). Safe Reinforcement Learning via Shielding. AAAI. arXiv:1708.08611.

- 14.** Fakih, M. et al. (2024). LLM4PLC. ICSE-SEIP 2024. arXiv:2401.05443.
- 15.** IEC 61508; IEC 61511; ISO 13849-1; IEC 62443 — функциональная безопасность и защита АСУ ТП.
- 16.** Регламент (ЕС) 2024/1689 (закон ЕС об ИИ), ст. 6 и Приложение III.
artificialintelligenceact.eu/article/6